

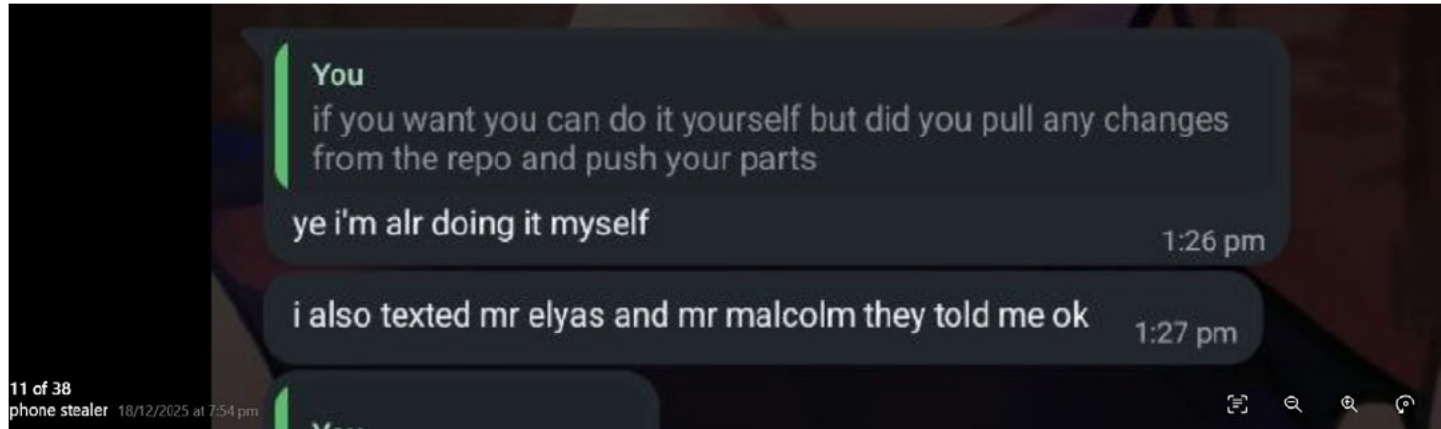
itd assignment 1 demo

pokkat

mark, arwen
itd oct25 asg1

this is me and arwens work
I dont know what happened

but please treat me and
arwens work as our owns
or at least give arwen marks



but still excuse darren and arwen
this is not their faults

Review Assignment Submission

File submission successful

Submission ID

9200686

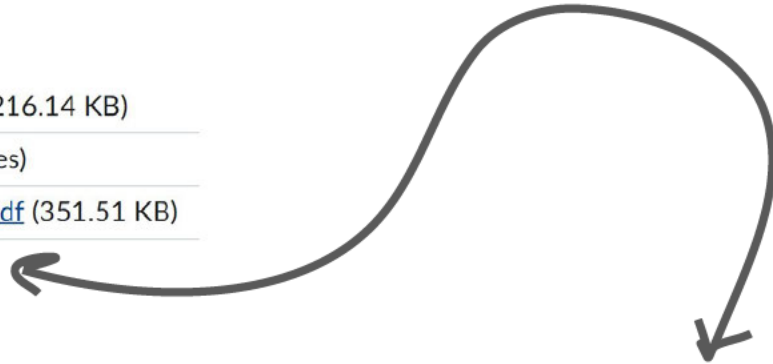
Submission(s)

 [ITD_Asg1_FeatureChecklist.pdf](#) (216.14 KB)

 [ITD_Asg1_GithubLink.txt](#) (39 Bytes)

 [ITD_Asg1_Pokkat_Walkthrough.pdf](#) (351.51 KB)

 [ITD_Asg1_Pokkat.apk](#) (69.7 MB)



Comments

 ITD_Asg1_Pokkat+ui_works+statskeeper_override_disabled.apk	24/12/2025 12:44 am	APK File	71,417 KB
---	---------------------	----------	-----------

Assignment

Assignment 1

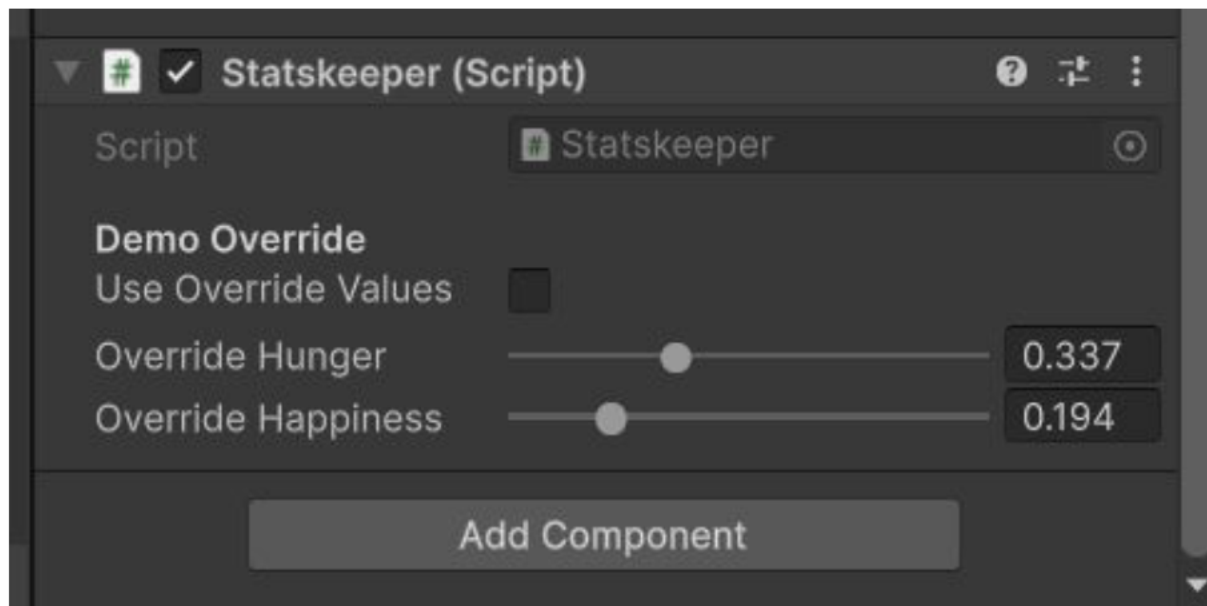
Submission Date

23 December 2025 23:59

This second apk has the settings page working and does not have the stats overridden for demo purposes

Other than that no core gameplay mechanic code was changed

Scenes/CoreGameplay
Pokkat Core (Game Object) > StatsKeeper (Component)





yes works in editor (compiles)

simulated image tracking however does not
(see walkthrough/readme pdf)

but image tracking DOES work when compiled

<https://datashower.joshwel.co/public/pokkat/xrsimenv.mp4>

<https://datashower.joshwel.co/public/pokkat/Screenshot%202025-12-22%20225926.png>

B2

Application must have a clear start (e.g. a start menu for users to start the experience)



start menu present



Yes we have a
start menu

B3

Application must have a clear end (e.g. player is informed they have completed the experience and prompted to return to start screen)



"the cat is satiated" to tell the user they can end

B6

There must be an overarching objective for users (e.g. scan all image targets, collect all objects, etc.)



prompts hint what to do

Event function Mark Joshwel

```
private void Start()
```

```
{
```

```
Configure_SubscribeToEvents();
```

```
Logkat.Out( message: "CoreGameplay: Start/Configure OK");
```

```
gameState = CoreGameplayState.WaitingForAnything;
```

```
private void OnImageDetected(HandledTrackedImage tracked)
```

```
{
```

```
Logkat.Dev(
```

```
message: $"CoreGameplay: OnImageDetected - pos={tracked.Image.tran
```

```
// update game state
```

```
switch (gameState)
```

```
{
```

```
case CoreGameplayState.WaitingForAnything:
```

```
gameState = CoreGameplayState.HasTrackerWaitingForPlane;
```

```
break;
```

```
case CoreGameplayState.HasPlaneWaitingForTracker:
```

```
gameState = CoreGameplayState.Ok;
```

```
break;
```

```
case CoreGameplayState.HasTrackerWaitingForPlane:
```

```
case CoreGameplayState.NekoWaitingForPlanes:
```

```
case CoreGameplayState.Ok:
```

```
break;
```

```
default:
```

```
Logkat.Panic( message: "unreachable game state");
```

```
break;
```

PokkatCore.CoreGameplay

```
// different prompt text for different neko states
```

```
⚡ Frequently called 1 usage 🐼 windy0ws +1
```

```
private string PromptText(CoreGameplayState state)
```

```
{
```

```
switch (state)
```

```
{
```

```
case CoreGameplayState.WaitingForAnything:
```

```
case CoreGameplayState.HasPlaneWaitingForTracker:
```

```
return "Scan the Pokkat tracker to spawn a cat!";
```

```
case CoreGameplayState.HasTrackerWaitingForPlane:
```

```
return "Move your phone around to detect surfaces!";
```

```
case CoreGameplayState.NekoWaitingForPlanes:
```

```
return "Move your phone around to detect more surfaces!";
```

```
case CoreGameplayState.Ok:
```

```
return
```

```
"Tap anywhere on your screen to place a bowl!\n"
```

```
+ "Tap your cat to pet and increase happiness!";
```

```
case CoreGameplayState.OkSatiated:
```

```
return "The cat is satiated, you can come back again later!";
```

```
default:
```

```
return "";
```

```
}
```

```
}
```

CoreGameplayUIManager



B4

Application must use Image Tracking.



PokkatCore.ImageHandling

```
public class ImageHandling : MonoBehaviour
```

```
/// 
```

```
/// <param name="args">event payload from the ar tracked image manager</param>
```

```
2 usages 2 Mark Joshwel
```

```
private void OnTrackablesChanged(ARTrackablesChangedEventArgs<ARTrackedImage> args)
```

```
{  
    // log the counts of added, updated, and removed trackables for debugging  
    Logkat.Dev(  
        message: $"ImageHandling: Trackables changed -> added:{args.added.Count} updated:  
  
    // process newly detected images (first time camera sees the image)  
    // args.added is a List<ARTrackedImage>  
    foreach (var trackedImage in args.added)  
    {  
        Logkat.Dev(message: $"ImageHandling: ... added: img={trackedImage.referenceImage}.  
        ProcessTrackedImage(trackedImage);  
    }  
  
    // process images with updated tracking information (pose, state changes)  
    // args.updated is a List<ARTrackedImage>  
    foreach (var trackedImage in args.updated)  
    {  
        Logkat.Dev(message: $"ImageHandling: ... updated: img={trackedImage.referenceImage}.  
        ProcessTrackedImage(trackedImage);  
    }  
}
```

PokkatCore.ImageHandling

B4

Application must use Image Tracking.



PokkatCore.ImageHandling

```
private void ProcessTrackedImage(ARTrackedImage trackedImage)
{
    // extract reference name from the image library entry
    var referenceName = trackedImage.referenceImage.name;

    // get current tracking state (Tracking, Limited, or None)
    var state = trackedImage.trackingState;

    // only fire detection event when image is actively being tracked
    // (TrackingState.Tracking means ar foundation has a good pose estimate)
    // (TrackingState.Limited or None are ignored - not reliable enough)
    if (state != TrackingState.Tracking) return;

    // log which image was detected with its id and state
    Logkat.Out(
        message: $"ImageHandling: Image detected '{referenceName}' ({trackedImage.trackableId}) state:{state}");

    // fire the OnImageDetected event with wrapped data
    // (wrap in HandledTrackedImage struct for consistent event signature)
    OnImageDetected?.Invoke(new HandledTrackedImage
    {
        State = state,
        Image = trackedImage,
        Id = trackedImage.trackableId
    });
}
```

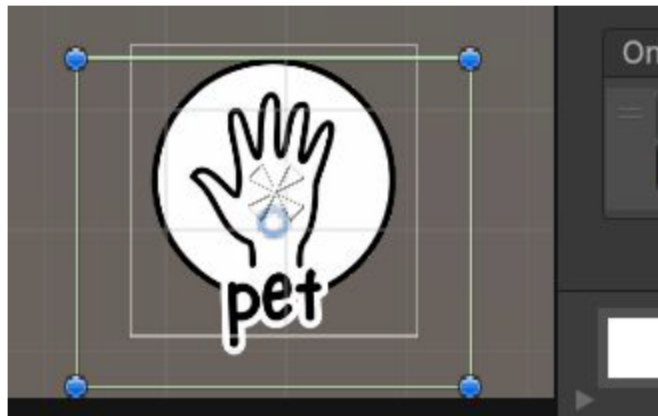
PokkatCore.ImageHandling

B5

Spawned objects from Image Tracking must be interactable through UI.



pet button



On Click

Editor And Run ▾

CoreGameplayUIManager.OnPetButton ▾

UI Manager ○



1 asset usage windyOws

```
public void OnPetButton()
```

```
{
```

```
Logkat.Dev( message: "CoreGameplayUIManager: pet");
```

```
coreGameplay.PetNeko();
```

```
}
```

Usages of 'OnPetButton()'

1 usage



Solution ▾

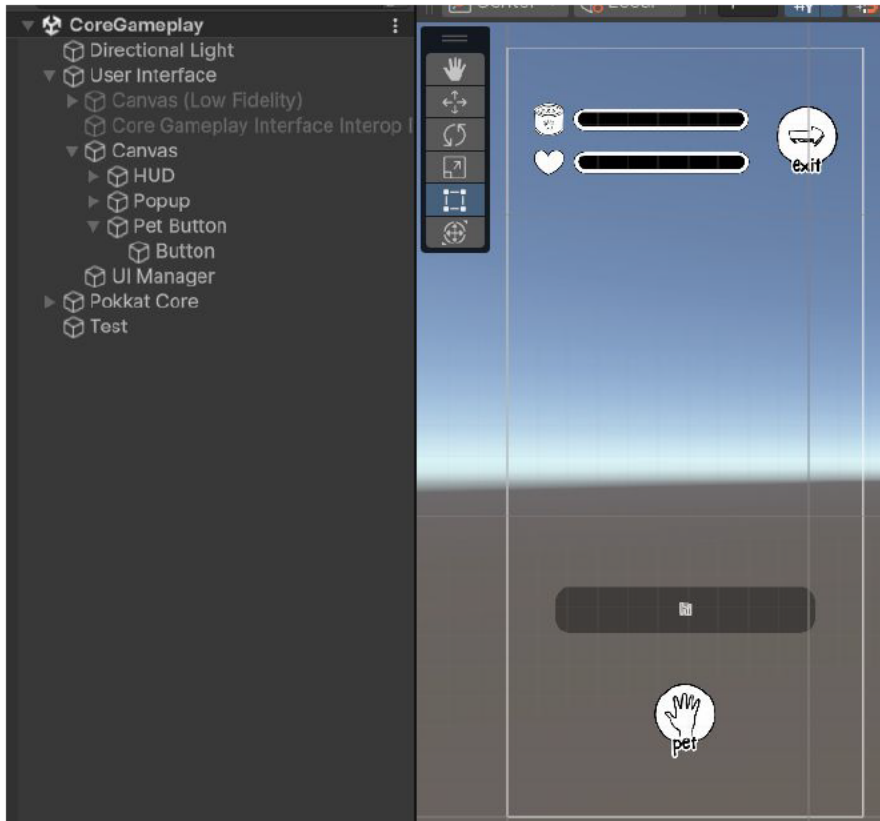


CoreGameplay.unity



User Interface/Canvas/Pet Button/Button

B7	There must be some indication on player progress (e.g. UI on screen)	✓	gameplay hud: prompts + stat bars
B8	There must be some feedback when player has completed a task (e.g. collected 1/5 objects, scanned 1/3 targets, etc.)	✓	gameplay hud: stat bars



```
public class CoreGameplayUIManager : MonoBehaviour
```

```
/// <Summary>
```

```
Event function & windy0ws +1 *
```

```
void Update()
```

```
{
```

```
// don't do anything if core gameplay isn't found
```

```
if (!coreGameplay || !coreGameplay.stats) return;
```

```
hungerFill.fillAmount = coreGameplay.stats.hunger;
```

```
happinessFill.fillAmount = coreGameplay.stats.happiness;
```

```
// updating user instruction prompts
```

```
string prompt = PromptText(coreGameplay.gameState);
```

```
promptText.text = prompt;
```

```
//dont show prompt panel if no prompts
```

```
bool showPanel = !string.IsNullOrEmpty(prompt);
```

```
promptText.transform.parent.gameObject.SetActive(showPanel);
```

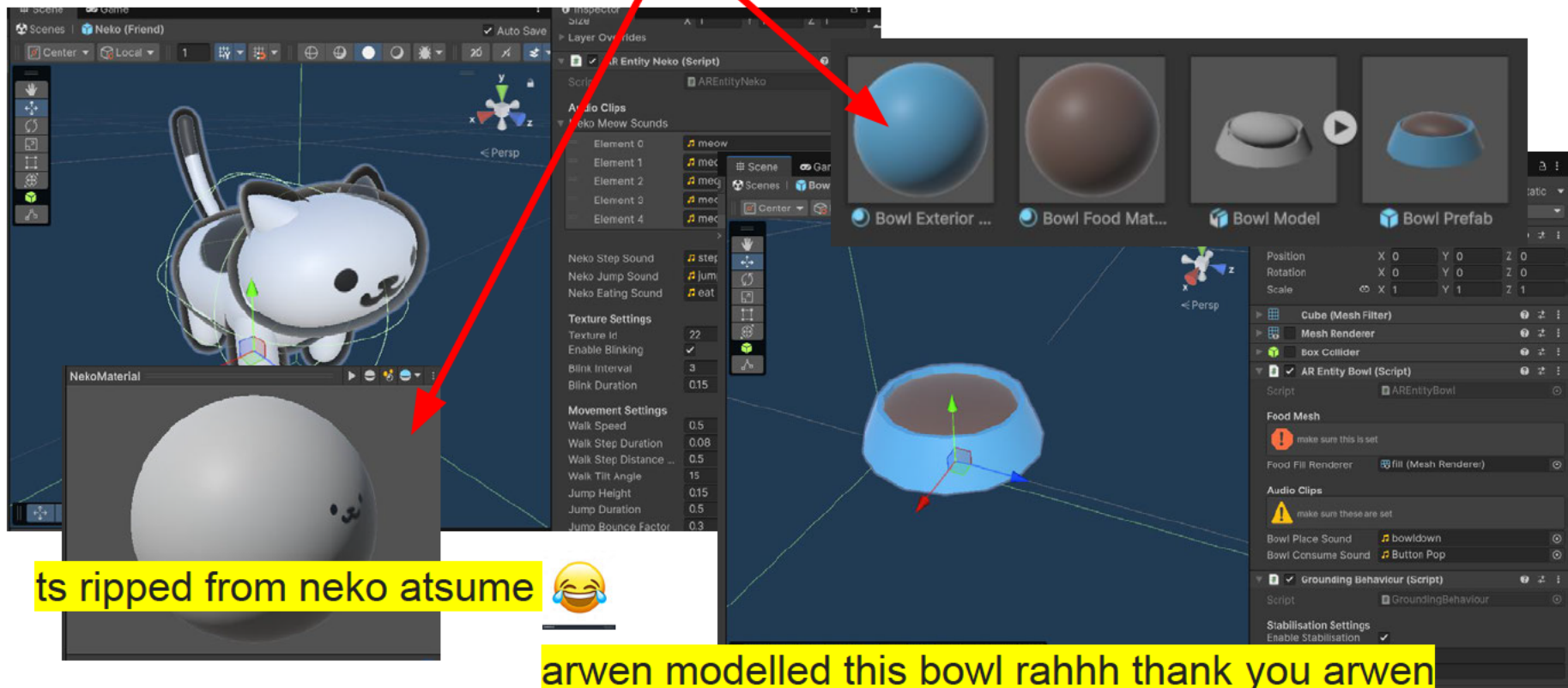
```
// show pet button only when main neko is spawned
```

```
if (petButton) petButton.SetActive(coreGameplay.hasMainNeko);
```

```
}
```

CoreGameplayUIManager

B9	Models used must not just be primitive shapes in Unity (e.g. Cube, Capsule, Cylinder, etc.)	✓	neko and bowl models
BX	3D Models must be textured	✓	neko and bowl models

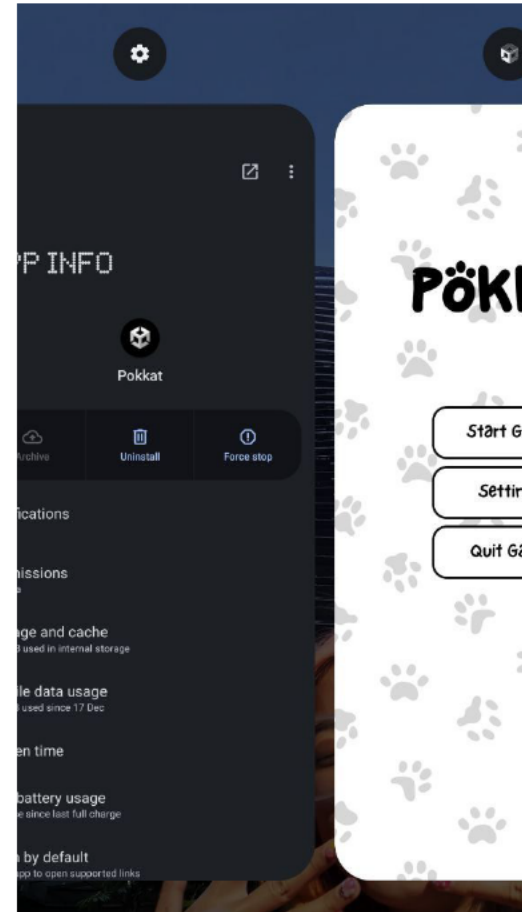
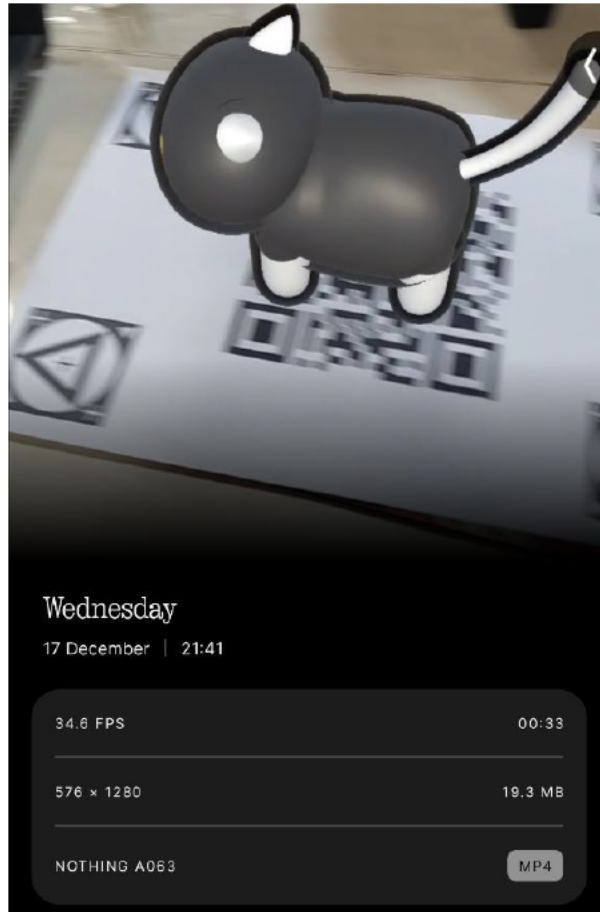


A1

The application can be deployed to a mobile device.



yes





2 usages Mark Joshwel

```
private void OnTrackablesChanged(ARTrackablesChangedEventArgs<ARPlane> args)
{
    Logkat.Dev(
        message: $"PlaneHandling: Trackables changed -> added:{args.added.Count} updated:{args

// skip threshold check if we've already fired and fireOnceOnly is enabled
if (fireOnceOnly && isReady) return;

// calculate total area of all tracked planes
var totalArea :float = CalculateTotalPlaneArea();

// bail out if we haven't reached the minimum area threshold
if (totalArea < minimumAreaSquareMeters) return;

// find the largest plane to pass to subscribers
var largestPlane = FindLargestTrackingPlane();

// bail out if no valid plane is found (shouldn't happen if totalArea > 0)
if (!largestPlane) return;

// mark as ready and fire the event
isReady = true;
Logkat.Out(
    message: $"PlaneHandling: Plane ready, total area {totalArea:F2}m², largest plane {largestPlane}
OnPlaneReady?.Invoke(largestPlane);
```

A2

The application incorporates additional AR features, such as plane detection or face tracking.



PokkatCore.PlaneHandling

```
/// <summary>
///     per-frame touch detection and plane interaction raycasting.
///     petting nekos has priority over plane interactions (bowl spawning)
/// </summary>
```

Event function 2 Mark Joshwel +1

```
private void Update()
```

```
{
    // try to get a touch position from this frame
    if (!TryGetTouchPosition(out var touchPosition :Vector2 )) return;

    // UI has highest priority - skip if touch is over a UI element (button, etc.)
    if (IsTouchOverUI())
    {
        Logkat.Dev( message: "PlaneHandling: touch is over UI, skipping plane/neko interaction")
        return;
    }

    // petting nekos has priority - check first before plane interactions
    if (TouchHitsNeko(touchPosition))
    {
        Logkat.Dev( message: "PlaneHandling: touch hit neko, skipping plane interaction");
        return;
    }

    // skip plane interaction if no subscribers are listening
    if (OnPlaneInteraction == null) return;

    // raycast from touch position to find plane hits
    if (!TryRaycastToPlane(touchPosition, out var interaction)) return;

    // fire the interaction event for subscribers (e.g., CoreGameplay)
    OnPlaneInteraction.Invoke(interaction);
}
```

PokkatCore.PlaneHandling

PokkatCore.CoreGameplay

```
public class CoreGameplay : MonoBehaviour
{
    private void OnPlaneInteraction(HandledPlaneInteraction interaction)

    // prevent spawning bowl too close to existing nekos
    // note: we pass the interaction position, not checking against existing bowl
    // because we're about to destroy the existing bowl anyway
    if (IsTooCloseToExistingEntities(interaction.Position))
    {
        Logkat.Out( message: "CoreGameplay: bowl spawn position too close to neko, skipping spawn");
        return;
    }

    if (_activeBowl)
    {
        Logkat.Out( message: "CoreGameplay: destroying existing bowl for replacement");
        Destroy(_activeBowl.gameObject);
        _activeBowl = null;
    }

    var spawned :GameObject = planeHandling.SpawnClosest(bowlPrefab, interaction.Position, playerCamera);
    if (!spawned) return;

    _activeBowl = spawned.GetComponent<AREntityBowl>();
    if (!_activeBowl)
    {
        Logkat.Warn("CoreGameplay: spawned bowl has no AREntityBowl component");
        Destroy(spawned);
        return;
    }

    Logkat.Out( message: $"CoreGameplay: spawned bowl at {interaction.Position}");
    // note: AREntityNeko listens for OnBowlSpawned directly
}
```

A2

The application incorporates additional AR features, such as plane detection or face tracking.



PokkatCore.PlaneHandling

```
var spawned :GameObject = planeHandling.SpawnClosest(bowlPrefab, interaction.Position, playerCamera);  
if (!spawned) return;
```

PokkatCore.CoreGameplay

```
public class PlaneHandling : MonoBehaviour  
{  
    public GameObject SpawnClosest(  
        // calculate rotation so the spawned object faces the camera  
        var toCamera :Vector3 = playerCamera.transform.position - spawnPosition;  
        var projectedForward :Vector3 = Vector3.ProjectOnPlane( vector: toCamera, targetPlane.normal).normalized;  
  
        // fallback if camera is directly above (projected forward is zero)  
        if (projectedForward.sqrMagnitude < 0.001f)  
            projectedForward = Vector3.ProjectOnPlane(Vector3.forward, targetPlane.normal).normalized;  
  
        var spawnRotation :Quaternion = Quaternion.LookRotation(projectedForward, upwards: targetPlane.normal);  
  
        // instantiate and parent to the plane so it moves with AR tracking adjustments  
        var spawned :GameObject = Instantiate(objectToSpawn, spawnPosition, spawnRotation);  
        spawned.transform.SetParent(targetPlane.transform, worldPositionStays: true);  
  
        Logkat.Out( message: $"PlaneHandling.SpawnClosest: spawned {objectToSpawn.name} at {spawnPosition}");  
        return spawned;  
    }  
}
```

PokkatCore.PlaneHandling

A3	AR objects within the application can interact with one another.	✓	main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned
A4	AR objects are aware when other AR objects appear.	✓	main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned

Find in Files 7 matches in 1 file

File mask: *.cs

Q OnNekoSpawned

Solution Project Directory Scope

Include non-solution items

```

public static event Action<AREntityNeko> OnNekoSpawned;
OnNekoSpawned += OnNekoSpawnedHandler;
OnNekoSpawned?.Invoke(this);
OnNekoSpawned -= OnNekoSpawnedHandler;
private void OnNekoSpawnedHandler(AREntityNeko otherNeko)

```

AREntityNeko.cs 185
 AREntityNeko.cs 231
 AREntityNeko.cs 233
 AREntityNeko.cs 254
 AREntityNeko.cs 730

entirely within PokkatCore.AREntityNeko

no CoreGameplay or GameManager coordination

A3	AR objects within the application can interact with one another.	✓	main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned
A4	AR objects are aware when other AR objects appear.	✓	main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned

```

/// <summary>
///     static event fired when any neko is spawned (for direct AR object awareness)
/// </summary>

```

Mark Joshwel

```
public static event Action<AREntityNeko> OnNekoSpawned;
```

Event function Mark Joshwel

```
private void Start()
```

```
{
```

```
    ApplyTexture(_eyesOpenTexture);
```

```
    if (enableBlinking) StartCoroutine(BlinkRoutine());
```

```
    AREntityBowl.OnBowlSpawned += OnBowlSpawnedHandler;
```

```
    OnNekoSpawned += OnNekoSpawnedHandler;
```

```
    OnNekoSpawned?.Invoke(this);
```

```
    StartC
```

```
}
```

```
/// <summa
```

```
public static event Action<AREntityNeko> OnNekoSpawned { add; remove; }
in class PokkatCore.AREntityNeko
```

static event fired when any neko is spawned (for direct AR object awareness)



entirely within PokkatCore.AREntityNeko

no CoreGameplay or GameManager coordination

A3	AR objects within the application can interact with one another.	✓	main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned
A4	AR objects are aware when other AR objects appear.	✓	main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned

Find in Files 10 matches in 3 files

File mask: *.cs

OnBowl

Solution Project Directory Scope

Include non-solution items

```

public static event Action<AREntityBowl> OnBowlSpawned;
public static event Action<AREntityBowl> OnBowlDestroyed;
OnBowlSpawned?.Invoke(this);
OnBowlDestroyed?.Invoke(this);
AREntityBowl.OnBowlSpawned += OnBowlSpawnedHandler;
AREntityBowl.OnBowlSpawned -= OnBowlSpawnedHandler;
private void OnBowlSpawnedHandler(AREntityBowl bowl)
// note: AREntityNeko listens for OnBowlSpawned directly

```

AREntityBowl.cs 65
 AREntityBowl.cs 70
 AREntityBowl.cs 117
 AREntityBowl.cs 131
 AREntityNeko.cs 230
 AREntityNeko.cs 253
 AREntityNeko.cs 717
 CoreGameplay.cs 390

entirely within PokkatCore.AREntityNeko and PokkatCore.AREntityBowl

no CoreGameplay or GameManager coordination

A3	AR objects within the application can interact with one another.		main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned
A4	AR objects are aware when other AR objects appear.		main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned

```
public sealed class AREntityBowl : MonoBehaviour
```

```
#region Static Events
```

```
/// <summary>
```

```
///     static event fired when any bowl is spawned (for direct AR object awareness)
```

```
/// </summary>
```

```
Ⓔ Mark Joshwel
```

```
public static event Action<AREntityBowl> OnBowlSpawned;
```

```
public sealed class AREntityBowl : MonoBehaviour
```

```
private void Start()
```

```
// broadcast spawn event for direct AR object awareness
```

```
OnBowlSpawned?.Invoke(this);
```

```
Logkat.Out( message: "AREntityBowl: broadcasted spawn event");
```

entirely within PokkatCore.AREntityNeko and PokkatCore.AREntityBowl

no CoreGameplay or GameManager coordination

A3	AR objects within the application can interact with one another.		main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned
A4	AR objects are aware when other AR objects appear.		main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned

```
/// <param name="otherNeko">the spawned neko</param>
```

2 usages Mark Joshwel

```
private void OnNekoSpawnedHandler(AREntityNeko otherNeko)
```

```
{
```

```
    // ignore self
```

```
    if (otherNeko == this) return;
```

```
    // only main neko initiates play with friends
```

```
    if (!CompareTag("NekoMain")) return;
```

```
    if (!_grounding.isGrounded) return;
```

```
    // only react to friend nekos
```

```
    if (!otherNeko.CompareTag("NekoFriend")) return;
```

```
    Logkat.Out( message: $"AREntityNeko: detected friend neko spawn at {otherNeko.transfo
```

```
    // queue friend for play (handles multiple friends spawning in quick succession)
```

```
    _pendingFriendQueue.Enqueue(otherNeko);
```

```
}
```

PokkatCore.AREntityNeko

A3	AR objects within the application can interact with one another.		main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned
A4	AR objects are aware when other AR objects appear.		main neko plays with friend neko when spawned; neko moves to bowl when bowl is spawned

```

/// <summary>
///     notifies main neko that a bowl has been placed - triggers move-and-eat behaviour
/// </summary>
// ReSharper disable once MemberCanBePrivate.Global
1 usage 2 Mark Joshwel
public void StateNotifyBowlPlaced()
{
    if (!CompareTag("NekoMain")) return;

    // cancel any existing move-and-eat coroutine (prevents stuck walking animation)
    if (_moveAndEatCoroutine != null)
    {
        StopCoroutine(_moveAndEatCoroutine);
        _moveAndEatCoroutine = null;
        ResetTilt();
    }

    _moveAndEatCoroutine = StartCoroutine( routine: MoveAndEat());
}

```

PokkatCore.AREntityNeko

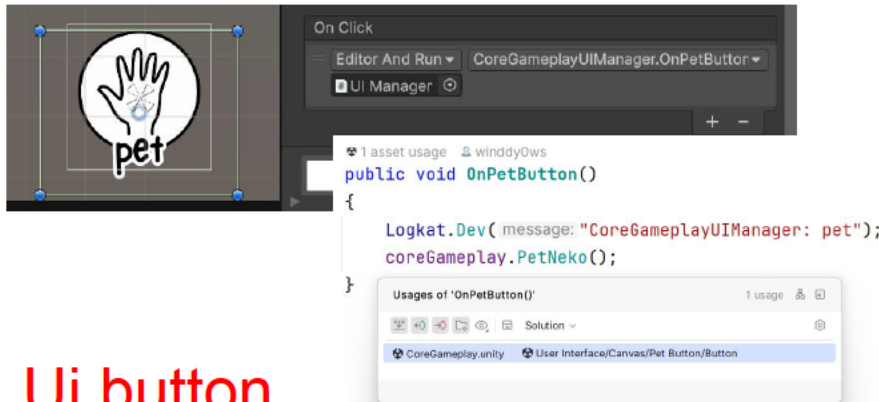
A5

The application has various forms of interaction between the player and AR elements.



pet via touch or ui button

B5 Spawned objects from Image Tracking must be interactable through UI.  pet button



Ui button

CoreGameplayUIManager

touch

PokkatCore.PlaneHandling

```
public class PlaneHandling : MonoBehaviour
{
    private static bool TouchHitsNeko(Vector2 screenPosition)
    {
        var ray = mainCamera.ScreenPointToRay(screenPosition);
        if (!Physics.Raycast(ray, out var hit, maxDistance: 100f))
        {
            Logkat.Dev(message: "PlaneHandling: TouchHitsNeko raycast hit nothing");
            return false;
        }

        Logkat.Dev(message: $"PlaneHandling: TouchHitsNeko raycast hit {hit.transform.name} (tag={hit.transform.tag})");

        // try to find neko component via tag or parent hierarchy
        var neko = hit.transform.CompareTag("NekoMain") || hit.transform.CompareTag("NekoFriend")
            ? hit.transform.GetComponent<AREntityNeko>() ?? hit.transform.GetComponentInParent<AREntityNeko>()
            : hit.transform.GetComponentInParent<AREntityNeko>();

        if (!neko)
        {
            Logkat.Dev(message: "PlaneHandling: TouchHitsNeko hit object has no AREntityNeko component");
            return false;
        }

        // trigger petting interaction on the neko
        Logkat.Out(message: $"PlaneHandling: petting neko {neko.name}");
        neko.Pet();
        return true;
    }
}
```

A6

The application can track multiple copies of the same image, with one object per image.



unique friend nekos are spawned when new image trackers are detected

(Unity) AR Foundation

ARTrackedImageManager.trackablesChanged

PokkatCore.ImageHandling

ImageHandling.OnTrackablesChanged

args.added / args.updated

ImageHandling.ProcessTrackedImage

state == Tracking

OnImageDetected event

```
public class ImageHandling : MonoBehaviour
{
    private void ProcessTrackedImage(ARTrackedImage trackedImage)
    {
        var referenceName = trackedImage.referenceImage.name;

        // get current tracking state (Tracking, Limited, or None)
        var state = trackedImage.trackingState;

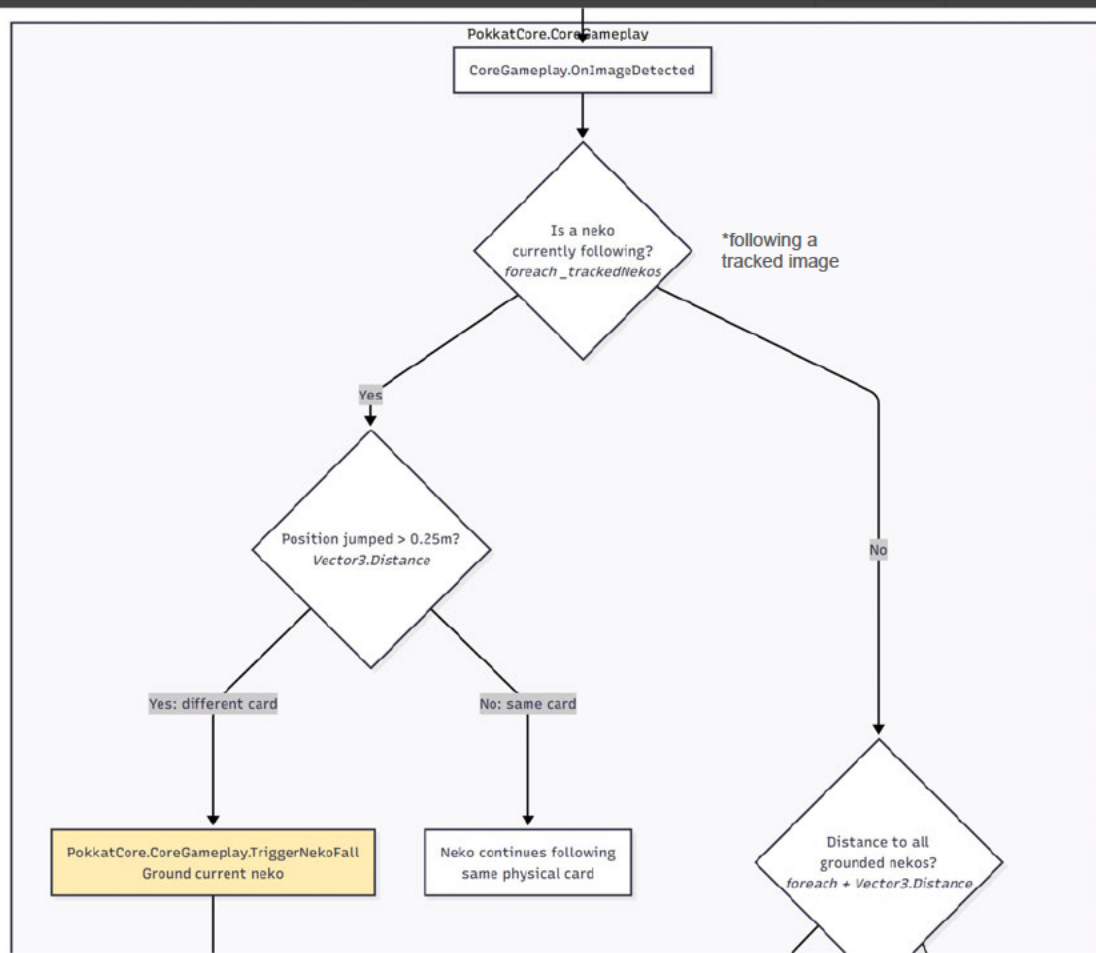
        OnImageDetected?.Invoke(new HandledTrackedImage
        {
            State = state,
            Image = trackedImage,
            Id = trackedImage.trackableId
        });
    }
}
```

PokkatCore.ImageHandling

The application can track multiple copies of the same image, with one object per image.



unique friend nekos are spawned when new image trackers are detected

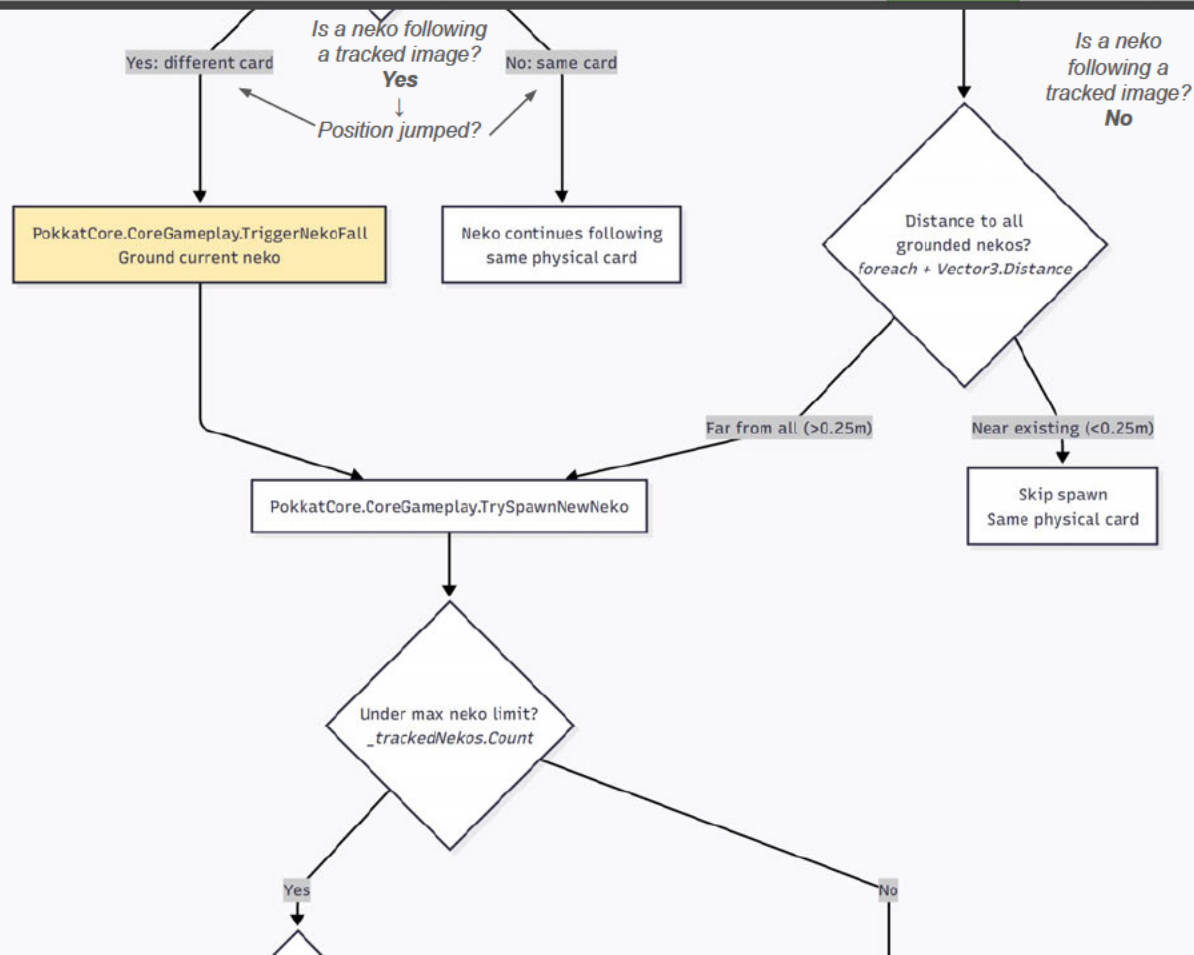


A6

The application can track multiple copies of the same image, with one object per image.



unique friend nekos are spawned when new image trackers are detected

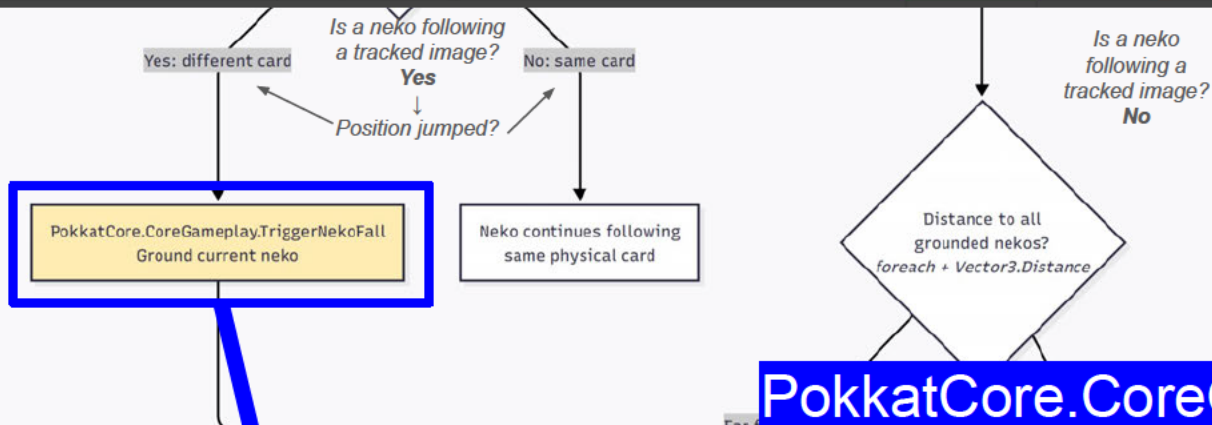


A6

The application can track multiple copies of the same image, with one object per image.



unique friend nekos are spawned when new image trackers are detected

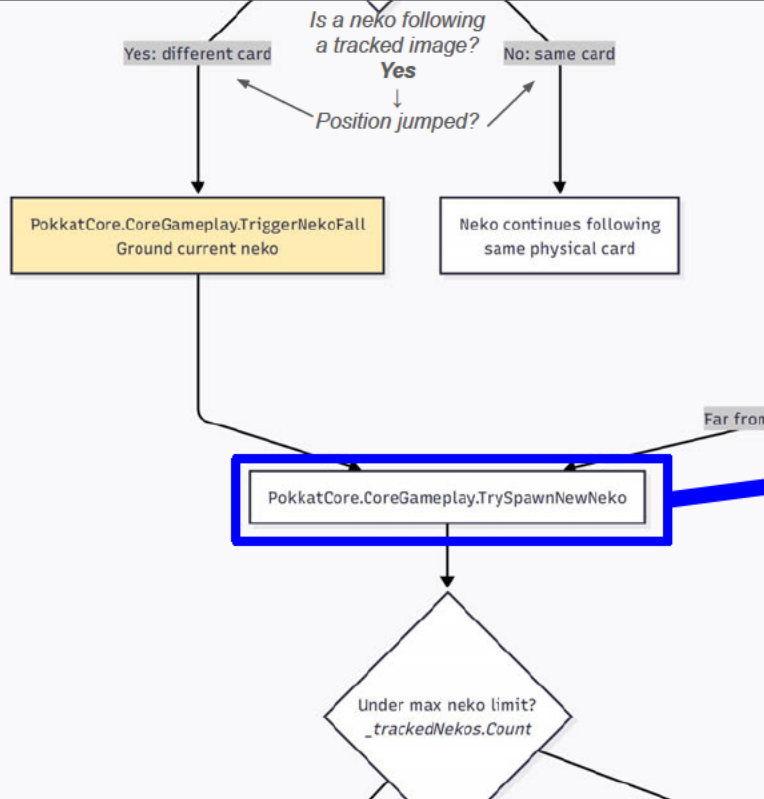


PokkatCore.CoreGameplay

```

private void TriggerNekoFall(TrackedNekoInstance neko)
{
    Logkat.Dev( message: $"CoreGameplay: TriggerNekoFall called, pos={neko.Entity.transform.position}");
    neko.IsFollowing = false;
    neko.Entity.StopFollowing();
    neko.Entity.Fall( onComplete: () =>
    {
        // callback when fall completes
        neko.IsGrounded = true;
        neko.AnchorPosition = neko.Entity.transform.position;
        Logkat.Dev( message: $"CoreGameplay: neko landed, anchorPosition={neko.AnchorPosition}");
    });
}
  
```

The application can track multiple copies of the same image, with one object per image.



PokkatCore.CoreGameplay

```

private void TrySpawnNewNeko(Vector3 position)
{
    // clean up any null entries (destroyed nekos)
    _trackedNekos.RemoveAll( match: a => a.TrackedNekoInstance == null || a.GameObject == null);
    // enforce spawn cooldown
    if (Time.time - _lastSpawnTime < spawnCooldown){...}
    // enforce max neko limit
    if (_trackedNekos.Count >= maxActiveNekos){...}

    // prevent spawning too close to existing entities (AR tracking blip protection)
    // ignoreFollowing=true because we're about to spawn a new following neko
    if (IsTooCloseToExistingEntities(position, ignoreFollowing: true)){...}

    // determine prefab
    var isMainNeko = !_mainNekoSpawned;
    var prefab :GameObject = isMainNeko ? mainNekoPrefab : friendNekoPrefab;
    if (!prefab){...}
    // calculate rotation to face the camera
    var toCamera :Vector3 = playerCamera.transform.position - position;
    toCamera.y = 0; // keep upright, only rotate on Y axis
    var rotation :Quaternion = toCamera.sqrMagnitude > 0.001f
        ? Quaternion.LookRotation(toCamera)
        : Quaternion.identity;

    // spawn at image position facing camera (will follow immediately)
    var spawned :GameObject = Instantiate(prefab, position, rotation);
    var nekoEntity = spawned.GetComponentInChildren<AREntityNeko>();
    if (!nekoEntity){...}

    // update spawn time
    _lastSpawnTime = Time.time;
    // create tracked instance
    var nekoInstance = new TrackedNekoInstance
    {
        Entity = nekoEntity,
        GameObject = spawned,
        AnchorPosition = position,
        IsFollowing = true,
        IsGrounded = false
    };

    _trackedNekos.Add(nekoInstance);
    nekoEntity.StartFollowing();

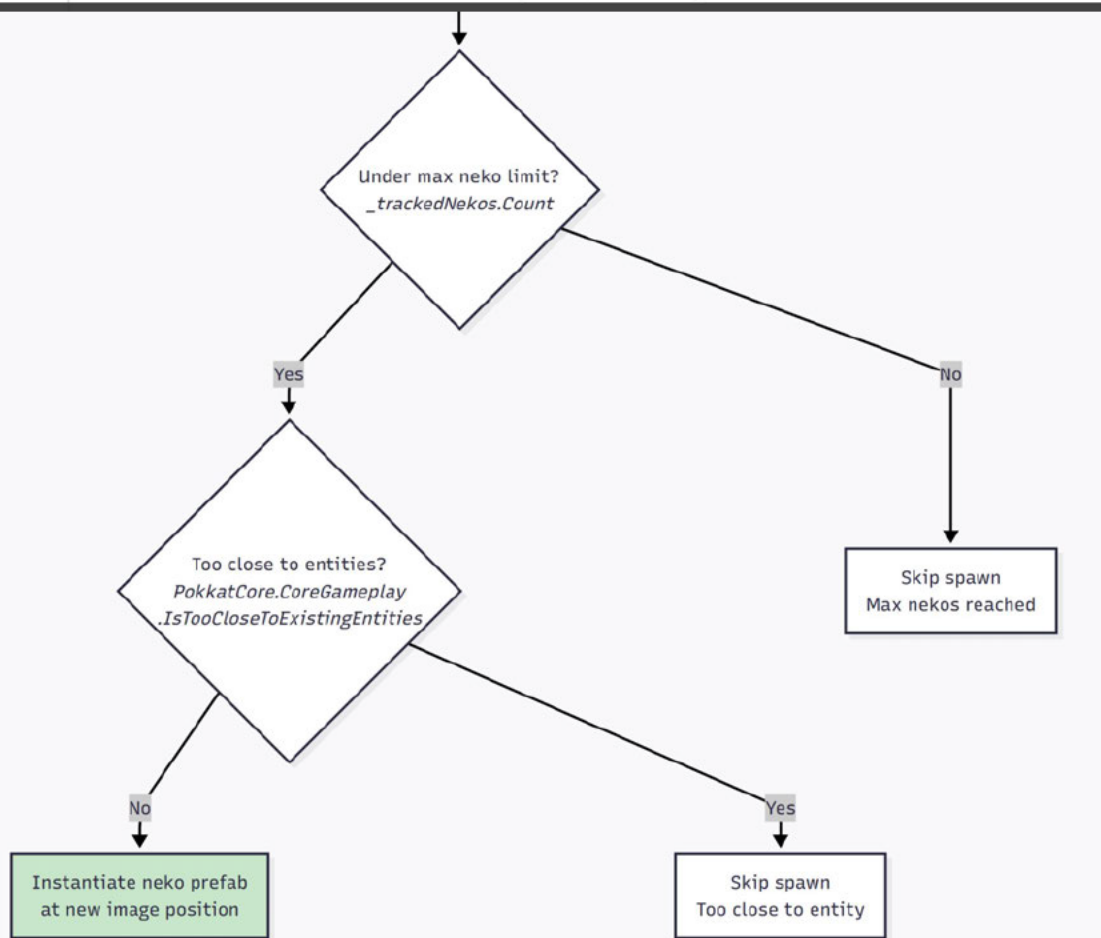
    if (isMainNeko)
    {
        _mainNekoSpawned = true;
        Logkat.Out( message: "CoreGameplay: spawned MAIN neko");
    }
    else {...}
}
  
```

A6

The application can track multiple copies of the same image, with one object per image.



unique friend nekos are spawned when new image trackers are detected

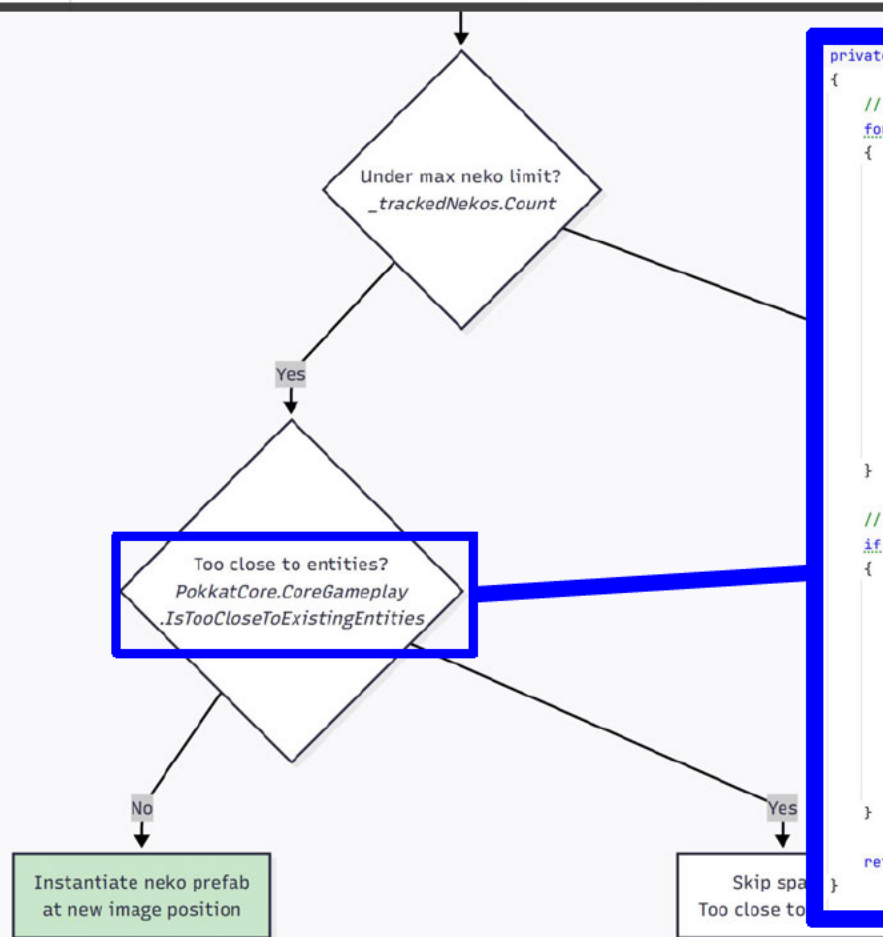


A6

The application can track multiple copies of the same image, with one object per image.



unique friend nekos are spawned when new image trackers are detected



```

private bool IsTooCloseToExistingEntities(Vector3 position, bool ignoreFollowing = false)
{
    // check against all grounded nekos
    foreach (var neko in _trackedNekos)
    {
        if (neko.Entity == null) continue;
        if (ignoreFollowing && neko.IsFollowing) continue;

        var nekoPos:Vector3 = neko.Entity.transform.position;
        var distance:float = Vector3.Distance(position, nekoPos);

        if (distance < entitySeparationDistance)
        {
            Logkat.Dev(
                message: $"CoreGameplay: position too close to neko ({distance:F3}m < {entitySeparationDistance}m)");
            return true;
        }
    }

    // check against active bowl
    if (_activeBowl != null)
    {
        var bowlPos:Vector3 = _activeBowl.transform.position;
        var distance:float = Vector3.Distance(position, bowlPos);

        if (distance < entitySeparationDistance)
        {
            Logkat.Dev(
                message: $"CoreGameplay: position too close to bowl ({distance:F3}m < {entitySeparationDistance}m)");
            return true;
        }
    }

    return false;
}
  
```

PokkatCore.CoreGameplay

U1	The design of menus is cohesive and consistent with the rest of the application.		technically
U3	UI elements are purposefully used and placed.		cher u decide
U4	The application considers mobile screen requirements.		



menu



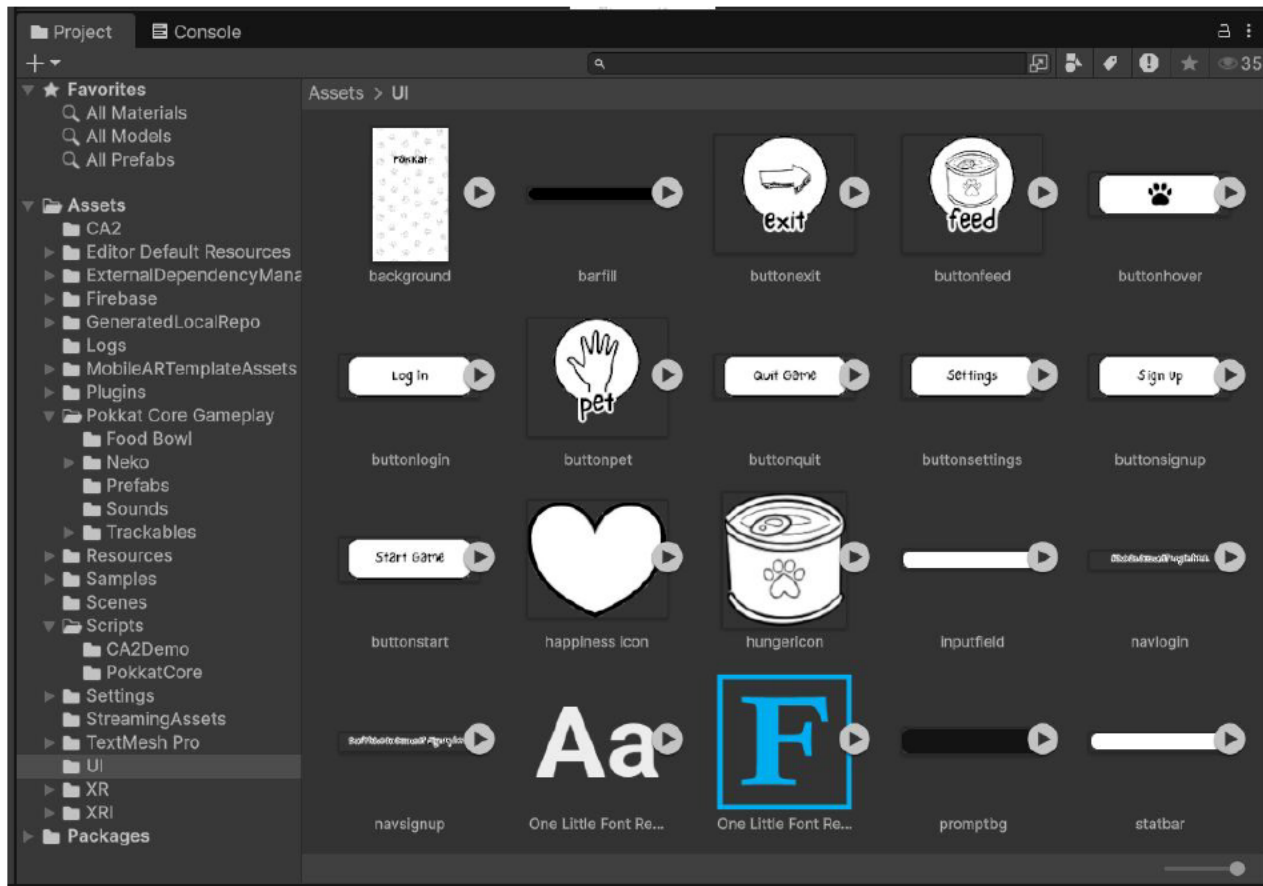
settings



gameplay



t.....yes.....



U4

The application considers mobile screen requirements.



(CoreGameplay Scene Only)



J1	Audio effects are effectively implemented within the game.		effective is questionable but there are audio effects
J2	The game includes feedback mechanisms.		neko jumps when you pet it
J3	The game responds appropriately to player actions.		cher u decide

(demo)